

42. Un damier

Le reste d'une division, et le motif apparaît.

1. Ce qu'il faut comprendre

`(x + y) % 2` vaut 0 une case sur deux, en quinconce — c'est la définition même d'un damier, et elle tient en trois caractères.

`a ? b : c` choisit en une expression. On aurait pu écrire un `if` sur quatre lignes ; ici la ligne dit « pose ceci ou cela », et c'est ce qu'on veut lire.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **956 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         77 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         736 o ← CopierTuiles - la police entière, et 2 tuiles dessinées
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1    34 o sur place, dans main() + AdresseCase (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

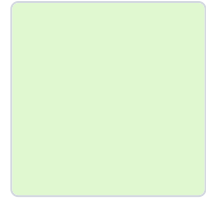
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : remplit tout l'écran d'un damier, cases claires et sombres.
//
// Deux tuiles : CLAIR (tout en .) et SOMBRE (tout en #).
// Une boucle dans une boucle parcourt toutes les cases de l'écran.
// La règle du damier : si colonne + ligne est pair → clair, sinon → sombre.
```

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
```

2. UN DESSIN — CLAIR

```
Tuile CLAIR = { ".....", ".....", ".....", ".....",  
               ".....", ".....", ".....", "....." };
```



à l'écran

3. UN DESSIN — SOMBRE

```
Tuile SOMBRE = { "#####", "#####", "#####", "#####",  
                "#####", "#####", "#####", "#####" };
```



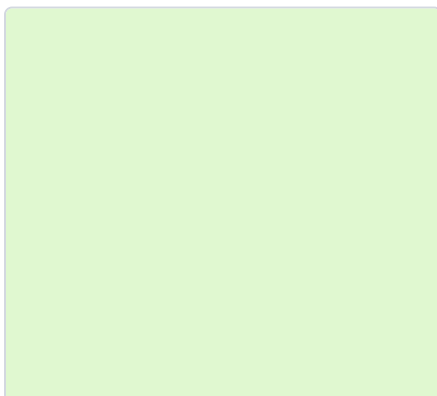
à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

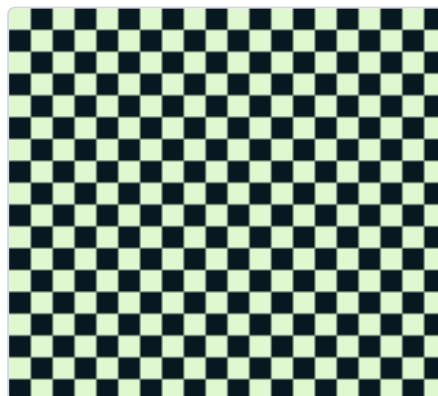
```
int main() {                                // Le jeu commence ici.  
    for (uint8_t y = 0; y < 18; y++) {      // chaque ligne y (0 à 17)  
        for (uint8_t x = 0; x < 20; x++) {   // chaque colonne x (0 à 19)  
            poser(x, y, (x + y) % 2 == 0 ? CLAIR : SOMBRE);  
                                                // (x + y) % 2 = le reste par 2 : 0 si pair.  
                                                // « condition ? A : B » = SI vrai A, SINON B.  
                                                // Exemple : x = 3, y = 5 → 8 % 2 = 0 → CLAIR.  
                                                //                x = 4, y = 5 → 9 % 2 = 1 → SOMBRE.  
        }  
    }  
  
    while (true) {                            // Répète pour toujours :  
        image();                             // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un damier sur tout l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ deux cases voisines différent
- ✓ une case sur deux revient
- ✓ et la rangée d’en dessous est décalée
- ✓ le damier couvre tout l’écran

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C’est le même texte, écrit autrement.

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

CLAIR

SOMBRE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d’origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

CLAIR

tuile n° 44 — poser(x, y, CLAIR)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

CLAIR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

CLAIR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

Ensuite : 43. Un lutin, hors de la grille — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++